Problem Set 3: Optimization and Motion Planning

6.4210/2 Robotic Manipulation — Fall 2026

Due: October 4, 2026

Out: September 24 **Due:** October 4

1 Bookwork

1.1 Differential kinematics

Problem 1 Pseudo-inverse [4]

1.1 For a redundant manipulator, which of the following expressions are well-defined at every robot configuration for any $\lambda > 0$? Select all that apply. The regularized expressions are known as "singularity robust" or "damped" inverses. [1]

Gradescope:

- ◯ $(J^\mathsf{T}J)^{-1}J^\mathsf{T}$
- ◯ $J^\mathsf{T}(JJ^\mathsf{T})^{-1}$
- √ $(J^\mathsf{T}J+\lambda I)^{-1}J^\mathsf{T}$
- √ $J^\mathsf{T}(JJ^\mathsf{T}+\lambda I)^{-1}$

Solution: *C and D. For a redundant arm J is 6×7, so $J^\mathsf{T}J$ is 7×7 with rank at most 6 — singular at* every *configuration, not just at singularities — and JJ^T loses rank when the arm is singular. Adding λI to either Gram matrix makes it positive definite, hence invertible everywhere. C and D are in fact the same matrix: both sides of $J^\mathsf{T}(JJ^\mathsf{T}+\lambda I)=(J^\mathsf{T}J+\lambda I)J^\mathsf{T}$ expand to $J^\mathsf{T}JJ^\mathsf{T}+\lambda J^\mathsf{T}$, and multiplying on the left by $(J^\mathsf{T}J+\lambda I)^{-1}$ and on the right by $(JJ^\mathsf{T}+\lambda I)^{-1}$ turns that into $(J^\mathsf{T}J+\lambda I)^{-1}J^\mathsf{T}=J^\mathsf{T}(JJ^\mathsf{T}+\lambda I)^{-1}$. The step needs $\lambda>0$, which is why A and B are not likewise a pair: at $\lambda=0$ the inverse in A does not exist.*

1.2 Redundancy is not the only reason to damp. Consider a two-joint arm whose Jacobian at some configuration is [3]

$$J=\begin{bmatrix}1 & 0\\ 0 & 0.001\end{bmatrix} \qquad \text{(meters per radian)},$$

It is square and the plain inverse exists. Now we ask for the end-effector velocity $V^d=(0,\ 1)^\top$ m/s.

Gradescope:

i. Using J^{-1}, the commanded joint velocity would be $\dot{q}=(0,$ **1000** $)$ rad/s.

ii. Using the damped form $J^\mathsf{T}(JJ^\mathsf{T}+\lambda I)^{-1}$ with $\lambda=0.01$, it is $\dot{q}=(0,$ **0.09999** $)$ rad/s. (Four significant figures.)

iii. That damped $\dot{q}$ is much smaller, but it does not achieve the requested end-effector velocity. The end-effector velocity it actually produces is $V=(0,$ **0.0001** $)$ m/s. (One significant figure.)

Solution: *J is diagonal, so everything decouples and only the second entry matters. The undamped gain is $1/\sigma=1/0.001=1000$. The damped gain is $\sigma/(\sigma^2+\lambda)=0.001/(10^{-6}+0.01)=0.09999$, four orders of magnitude smaller. Feeding that back through J gives $\sigma\cdot0.09999=9.999\times10^{-5}$ m/s against the 1 m/s requested — 0.0001 m/s to the one significant figure asked for. Damping does not change what the arm can do: J is invertible, and the arm can move in that direction, at a millimetre per radian of joint motion. It changes what the controller asks for — rather than command 1000 rad/s, it declines to try.*

Going further: The damped gain applied to a singular value σ is $\sigma/(\sigma^2+\lambda)$. For which value of σ is it largest, and how large does it get? Use that to say what a choice of λ is really choosing.

Solution: *Differentiating, the gain peaks at $\sigma = \sqrt{\lambda}$, where it equals $1/(2\sqrt{\lambda})$ — here $\sigma = 0.1$ and a gain of 5. So λ picks a ceiling on joint speed: no commanded task velocity of unit norm can ever be amplified by more than $1/(2\sqrt{\lambda})$. It also picks the width of the band it costs you: directions with $\sigma \gg \sqrt{\lambda}$ are followed essentially untouched, and directions with $\sigma \ll \sqrt{\lambda}$ are given up on. Small λ means accurate tracking closer to the singularity and a higher velocity ceiling when you get there.*

Problem 2 DiffIK via optimization [5]

2.1 In class, we have seen that we can formulate the differential IK problem as a constrained optimization problem. What advantages does the optimization approach (OPT) have over using a singularity-robust pseudo-inverse (SRPI) on its own? [1]

Gradescope:

- ◯ OPT produces reasonable robot velocities even near singularities whereas SRPI does not
- ◯ OPT will never cause the robot to stop before reaching its goal while SRPI can
- √ **OPT can enforce the robot's actual joint limits and SRPI can't**
- √ **OPT can enforce the robot's actual bounds on joint velocities while SRPI can't**
- √ **OPT can enforce the robot's actual bounds on joint accelerations while SRPI can't**

Solution: *The last three. The first is what damping already buys you — behaving well near singularities is precisely the point of SRPI. The second has it backwards: it is the optimization that can stop short. If reaching the target means moving a joint that is already at its limit, the QP's best feasible answer can be $\dot{q} = 0$, and the robot stalls with the target still out of reach, whereas SRPI always returns some velocity. (With acceleration limits the QP can even be infeasible: approaching a position limit too fast, no allowed deceleration keeps the joint inside it.) What optimization genuinely adds is the last three: the joint position, velocity and acceleration limits are written down as constraints, so its solution cannot violate them, while SRPI will happily command a velocity that drives a joint past its limit.*

2.2 It is tempting to think you could get the same effect by computing the pseudo-inverse solution and then clipping it into the limits. Here is a two-joint arm with a one-dimensional task, at a configuration where [4]

$$J = \begin{bmatrix} 1 & 2 \end{bmatrix}, \qquad V^d = 1 \text{ m/s}, \qquad |\dot{q}_i| \leq 0.3 \text{ rad/s for } i = 1, 2.$$

The optimization formulation is $\min_{\dot{q}} \left\| J\dot{q} - V^d \right\|^2$ subject to those bounds.

Gradescope:

i. The (undamped) pseudo-inverse solution is $\dot{q} = ($ **0.2** , **0.4** $)$ rad/s, which is infeasible.

ii. Clip each component of that answer into $[-0.3, 0.3]$ and push it back through J. The end-effector velocity you get is **0.8** m/s.

iii. Solve the optimization instead. It returns $\dot{q} = ($ **0.3** , **0.3** $)$ rad/s, giving an end-effector velocity of **0.9** m/s.

Solution: *$J^+ = J^\mathsf{T}(JJ^\mathsf{T})^{-1} = (1, 2)^\mathsf{T}/5 = (0.2, 0.4)$. Clipping gives $(0.2, 0.3)$ and $J\dot{q} = 0.2 + 0.6 = 0.8$. The QP wants $J\dot{q}$ as close to 1 as the box allows, and $\dot{q}_1 + 2\dot{q}_2$ is maximized over the box at the corner $(0.3, 0.3)$, where it equals 0.9. Clipping left joint 1 at 0.2 because that is what the pseudo-inverse asked for; the QP notices that joint 2 has run out of room and spends joint 1's remaining authority instead. Squared task error 0.04 versus 0.01.*

Going further: The task here was one-dimensional, so both answers at least pointed the right way and only the speed was wrong. With a six-dimensional V^d and a cost of $\left\| J\dot{q} - V^d \right\|^2$, what can the QP's answer do that you might not want? What would you change so it can't?

Solution: *Least squares is free to trade error in one task direction against error in another, so when the constraints bite it can return a $\dot{q}$ whose end-effector velocity points somewhere quite different from V^d — the gripper drifts sideways off the intended path rather than merely lagging along it. For a straight-line pick-and-place motion that is much worse than being slow. The standard fix is to insist on the direction and give up only the magnitude: solve $\max_{\alpha,\dot{q}} \alpha$ subject to $J\dot{q} = \alpha V^d$, $0 \le \alpha \le 1$, and the joint bounds. The end effector then always moves along V^d, just possibly slower than asked, which is what Drake's* `DifferentialInverseKinematics` *does.*

Problem 3 Joint centering [5]

Eq (23) in Chapter 3 of the notes describes how to do "joint centering" in the optimization formulation.

$$\begin{aligned} \min_{v_n} \quad & \left\| J^G(q) v_n - V^{G_d} \right\|_2^2 + \epsilon \left\| P(q) \left(v_n - K(q_0 - q) \right) \right\|_2^2, \\ \text{subject to} \quad & \text{constraints.} \end{aligned} \tag{23}$$

Here v_n is the next commanded joint velocity (equal to $\dot{q}$ for the fixed-base iiwa), and $0 < \epsilon \ll 1$ weights the joint-centering objective relative to end-effector tracking. The constraints are the joint position, velocity, and acceleration limits. The null-space projection is $P(q) = I - J^G(q)^+ J^G(q)$, where J^+ is the undamped Moore–Penrose pseudoinverse.

The "classic" formulation of a joint centering controller, not formulated using optimization, is:

$$\dot{q} = J^G(q)^+ V^{Gd} + P(q)\, K(q_0 - q).$$

3.1 Which of these are true for the iiwa? [1]

Gradescope:

- ◯ $\dot{q}$ is a 6-d spatial velocity
- √ $\dot{q}$ **is a 7-d joint velocity**
- √ V^{Gd} **is a 6-d spatial velocity**
- ◯ V^{Gd} is a 7-d joint velocity
- ◯ V^{Gd} depends on the chosen representation for rotation
- √ V^{Gd} **does not depend on the chosen representation for rotation**
- ◯ $J^G(q)$ is 7×6
- √ $J^G(q)$ **is** 6×7
- ◯ $K(q_0 - q)$ has units of radians
- √ $K(q_0 - q)$ **has units of radians/sec**
- ◯ $P(q)$ is 7×6
- ◯ $P(q)$ is 6×7
- ◯ $P(q)$ is 6×6
- √ $P(q)$ **is** 7×7
- √ $J^G(q)(P(q)v) = 0$
- ◯ $J^G(q)(P(q)v) \neq 0$

Solution: *$\dot{q}$ is a 7-d joint velocity; V^{Gd} is a 6-d spatial velocity and does not depend on the rotation representation (it is an angular velocity, not the derivative of a parameterization); $J^G(q)$ is 6×7; $K(q_0 - q)$ is intended to be a joint velocity, so radians/sec; $P(q) = I - J^+ J$ is 7×7; and $J^G(q)(P(q)v) = 0$, which is what makes the secondary task invisible to the end-effector.*

3.2 The classic controller is easy to understand for a two-joint arm with a one-dimensional task. Dropping the G superscript on J, take [4]

$$J = \begin{bmatrix} 1 & 1 \end{bmatrix}, \qquad V^{Gd} = 0.2, \qquad q = (0.6,\ 0), \qquad q_0 = (0,\ 0), \qquad K = 1.$$

Gradescope:

i. The primary term is $J^+V^{Gd} = (\underline{\quad \textbf{0.1} \quad}, \underline{\quad \textbf{0.1} \quad})$.
ii. The off-diagonal entry P_{12} of the projector is $\underline{\quad \textbf{-0.5} \quad}$.
iii. The secondary term is $PK(q_0 - q) = (\underline{\quad \textbf{-0.3} \quad}, \underline{\quad \textbf{0.3} \quad})$.
iv. Their sum is $\dot{q} = (\underline{\quad \textbf{-0.2} \quad}, \underline{\quad \textbf{0.4} \quad})$, and $J\dot{q} = \underline{\quad \textbf{0.2} \quad}$ — unchanged, which is the point.
v. Back on the real iiwa, away from singularities, the rank of $P(q)$ is $\underline{\quad \textbf{1} \quad}$.

Solution: *$J^+ = J^\mathsf{T}(JJ^\mathsf{T})^{-1} = (1,1)^\mathsf{T}/2$, so $J^+V^{Gd} = (0.1, 0.1)$ and*

$$P = I - J^+J = \begin{bmatrix} 0.5 & -0.5 \\ -0.5 & 0.5 \end{bmatrix}.$$

With $K(q_0 - q) = (-0.6, 0)$, the secondary term is $(-0.3, 0.3)$, and the sum is $(-0.2, 0.4)$ with $J\dot{q} = -0.2 + 0.4 = 0.2 = V^{Gd}$. On the iiwa J^G is 6×7 of rank 6 away from singularities, so $\operatorname{rank} P = 7 - 6 = 1$.

Going further: Why would you want to do joint centering at all? And look again at the numbers above: the secondary term drives joint 1 back toward its center but drives joint 2 *away* from its center. Is it still doing its job? Then think about the iiwa, where you just found the nullspace is one-dimensional — and about what happens to that number when the arm reaches a singularity.

Solution: *The end-effector task fixes only 6 of 7 numbers, so something has to choose the seventh; left to itself the arm drifts until a joint hits a limit and the task fails for reasons that have nothing to do with the task. Centering spends the spare freedom on staying away from limits, which also tends to keep you away from singular configurations and gives the next motion room to manoeuvre. It is still doing its job in the example: the objective is $\|q - q_0\|^2$, and, writing $\dot{q}_n = PK(q_0 - q)$ for the null-space part of $\dot{q}$ — the secondary term's own contribution to the motion — it changes at $\frac{d}{dt}\|q - q_0\|^2 = 2(q - q_0) \cdot \dot{q}_n = 2(0.6, 0) \cdot (-0.3, 0.3) = -0.36 < 0$. Gradient descent on a scalar objective is under no obligation to improve every coordinate. On the iiwa, though, the whole secondary command $K(q_0 - q)$ — seven numbers — gets projected onto a one-dimensional subspace, so you get one scalar's worth of it and no more; centering all seven joints at once is usually not available. When the arm reaches a singularity and J loses rank, the nullspace grows (rank P becomes 2 or more), giving more freedom for the secondary task. Merely approaching a singularity does not change the nullspace dimension, although the undamped primary command can become very large.*

Problem 4 Hold my beer [5]

The differential IK QP is re-solved at every control tick, with timestep h, and its decision variable is the joint velocity $\dot{q}$ for this tick only. Limits that are naturally about position or acceleration therefore have to be rewritten as bounds on $\dot{q}$. Throughout, $h = 0.01$ s and we look at a single joint with upper limit $q^{\max} = 1.57$ rad and velocity limit $|\dot{q}| \le 2.0$ rad/s.

4.1 A first-order guard against overshooting the position limit is to require that one timestep of $\dot{q}$ does not move you past it: $q + h\,\dot{q} \le q^{\max}$. [3]

Gradescope:

i. At $q = 1.50$ rad this bounds $\dot{q}$ above by $\underline{\quad \textbf{7.0} \quad}$ rad/s.
ii. At $q = 1.56$ rad it bounds $\dot{q}$ above by $\underline{\quad \textbf{1.0} \quad}$ rad/s.
iii. At $q = 1.50$, which gives the tighter upper bound on $\dot{q}$? ◯ position ✓ **velocity**
iv. At $q = 1.56$, which gives the tighter upper bound on $\dot{q}$? ✓ **position** ◯ velocity

Solution: *$\dot{q} \le (q^{\max} - q)/h$, which is $0.07/0.01 = 7.0$ and $0.01/0.01 = 1.0$. The QP takes the tighter of that and the velocity limit, so at $q = 1.50$ the velocity limit binds ($2.0 < 7.0$) and at $q = 1.56$ the position limit binds ($1.0 < 2.0$).*

4.2 An acceleration limit $|\ddot{q}| \leq 10$ rad/s^2 becomes a bound relative to the velocity the previous tick actually commanded: $|\dot{q} - \dot{q}_{\text{prev}}| \leq h\ |\ddot{q}|^{\max}$. Suppose $\dot{q}_{\text{prev}} = 0.50$ rad/s and the cost alone would like $\dot{q} = 1.2$ rad/s. [2]

Gradescope:

i. The acceleration limit confines $\dot{q}$ to the interval [**0.40** , **0.60**] rad/s.

ii. With q far from its limits, the QP therefore returns $\dot{q} =$ **0.60** rad/s.

Solution: *$h\ |\ddot{q}|^{\max} = 0.01 \times 10 = 0.1$, so $\dot{q} \in [0.40, 0.60]$. The cost is minimized as close to 1.2 as the box allows, which is the upper end, 0.60. It will take several ticks to ramp up to what the cost wants.*

Going further: The velocity limit is a statement about this tick alone; the other two bring in q and $\dot{q}_{\text{prev}}$, so the controller is no longer memoryless. Two things can now go wrong that could not before. First, suppose the joint is running at 2.0 rad/s straight at $q^{\max}$: does the one-step position constraint above actually keep it safe, given the acceleration limit? Second, what happens to the QP if q somehow ends up slightly *outside* its limit? In each case, what would you change?

Solution: *It does not keep it safe. Stopping from 2.0 rad/s at 10 rad/s^2 takes 0.2 s and covers 0.2 rad, but the one-step constraint is happy to run at full speed until 0.02 rad from the limit — by which point no feasible deceleration exists and the QP becomes infeasible with the joint still moving. You need a bound that accounts for the stopping distance, e.g. $\dot{q} \leq \sqrt{2\,|\ddot{q}|^{\max}\,(q^{\max} - q)}$, or more simply a conservative velocity envelope like $\dot{q} \leq -k(q - q^{\max})$ that ramps down over a whole braking region rather than one tick. Second: if $q > q^{\max}$ the constraint $\dot{q} \leq (q^{\max} - q)/h$ is negative, which is fine — it commands motion back toward the limit — but combined with a velocity or acceleration bound that cannot supply that much negative velocity, the feasible set is empty and the QP returns nothing at all. A controller that outputs nothing is worse than one that outputs something imperfect, so in practice the position constraint is softened: introduce a slack variable, allow $\dot{q} \leq (q^{\max} - q)/h + s$ with $s \geq 0$, and put a large penalty on s in the cost. The QP then stays feasible always and violates the limit only as much as it must.*

1.2 Motion planning

Problem 5 Search space [1]

Gradescope: Collision-free motion planning for robots involves searching for a path through the space of robot configurations. Which of the following are minimal representations sufficient for collision checking and goal specification under these assumptions? Assume the circular and cylindrical robots can translate freely in the plane, and their heading does not affect collision checking or the goal. The cylinder stays upright at a fixed height. Assume the robot arms have fixed bases and all their joints are revolute; any joint limits are respected in the allowed ranges of the joint-angle coordinates.

- √ **The (x, y) position of a circular robot moving in a plane.**
- √ **The (x, y) position of a cylindrical (holonomic) robot rolling on the floor of a 3D room.**
- ◯ The (x, y) position of the center of an arbitrary rigid robot moving in a plane.
- √ **The (x, y, θ) pose of an arbitrary rigid robot moving in a plane.**
- √ **The joint angles of a 6-dof robot arm.**
- ◯ The pose (position, quaternion) of the end-effector of a 6-dof robot arm.
- √ **The joint angles of a 7-dof robot arm.**
- ◯ The pose (position, quaternion) of the end-effector of a 7-dof robot arm.

Solution: *The first, second, fourth, fifth and seventh. Two terms: a robot's* heading *is the direction it faces, the angle θ of its body in the plane; a* holonomic *robot can move in any direction from any configuration, so its mechanics do not restrict which paths are possible. In order:*

- *Circular robot, (x, y): yes — heading changes neither its footprint nor the goal, so two numbers capture everything that matters.*
- *Cylindrical holonomic robot, (x, y): yes, likewise; it stays upright at a fixed height, so the 3D room adds nothing.*
- *Arbitrary rigid robot, (x, y) only: no — its footprint turns with it, so collisions depend on θ.*
- *Arbitrary rigid robot, (x, y, θ): yes.*
- *6-dof arm, joint angles: yes — with the base fixed they place every link, and joint limits just restrict their ranges.*
- *6-dof arm, end-effector pose: no — several configurations reach the same pose and differ in where the links are, so the pose cannot determine collisions along the rest of the linkage.*
- *7-dof arm, joint angles: yes, as for the 6-dof arm.*
- *7-dof arm, end-effector pose: no, and worse — a redundant arm has a whole family of configurations for one pose.*

Going further: A 7-joint arm can change its elbow position while keeping the same end-effector pose. Why might this matter when planning around a shelf?

Solution: *One elbow configuration may collide with the shelf while another is clear. End-effector pose alone cannot distinguish them; the planner needs the arm's configuration.*

Problem 6 Dimension [1]

Gradescope: Consider two Kuka iiwa 14 robots, each equipped with a WSG gripper, both rigidly mounted on a single cylindrical holonomic robot platform (1m tall, 1m diameter). Assume the platform moves on a flat floor and stays upright. Each gripper can open and close, and its two fingers are coupled to move symmetrically. What is the dimension of the minimal configuration space? Dimension: **19**

Solution: *19. The platform is cylindrical, but the arms are rigidly mounted on it, so rotating it moves them: the base contributes (x, y, θ), three dimensions rather than two. Each iiwa adds seven, and each WSG one (the two fingers are coupled), giving $3 + 2 \times 7 + 2 \times 1 = 19$.*

If the fingers were separately controllable, it would be 21.

Going further: Suppose both grippers are locked at fixed openings. Does this make collision-free planning necessarily easier, even though it reduces the configuration-space dimension?

Solution: *The dimension drops from 19 to 17, so there are fewer variables to search. But locking the grippers also removes possible motions: a passage that required closing a gripper may become impossible.*

Problem 7 Completeness and practicality [6]

Gradescope: Consider the following motion planning domains (assume all obstacles are either polygons in 2D or polyhedra in 3D):

1. 2D Translating polygons
2. 2D Rotating/Translating polygons
3. 3D Translating (in a plane) polyhedra
4. 3D Translating polyhedra
5. 3D Rotating/Translating polyhedra
6. 2-joint 2D robot

7. 2-joint 3D robot
8. 7-joint 2D robot
9. 7-joint 3D robot

Answer one of *none*, *all* or *some* for each of the questions below.

For practicality, consider typical problems requiring useful geometric accuracy, rather than specially easy instances. Judge whether the method is generally computationally feasible in each domain.

7.1 Grid search is *complete* for which of these problem domains? ___**none**___ *Resolution complete*? ___**all**___ A *practical* choice? ___**some**___ [3]

Solution: *Three terms.* Complete*: the algorithm finds a path whenever one exists, and reports that none exists when none does, in finite time.* Resolution complete*: at a fine enough grid it finds a path whenever one exists with positive clearance; failing at one resolution proves nothing, since a finer grid might still succeed. A* practical *choice: for typical problems in the domain, at the accuracy you need, it finishes in reasonable time and memory.*

Complete in none: a grid can step over a passage narrower than its cells, so a failure never proves that no path exists. Resolution complete in all: halve the cells repeatedly until they are smaller than the narrowest gap the path passes through (its clearance, as in Problem 11); every cell along the path is then free, so the search finds it. The intended practicality answer is "some": uniform grids are generally feasible in low-dimensional Cspaces, while useful resolution in six or seven dimensions is usually too expensive. Cspace dimension matters more than workspace dimension: a 2-joint arm in 3D still has only two configuration coordinates, whereas a free-flying polyhedron has six. The judgment also depends on the required accuracy, collision-checking cost and available resources.

7.2 Classic PRM/RRT algorithms are *complete* for which of these problem domains? ___**none**___ *Probabilistically complete*? ___**all**___ A *practical* choice? ___**all**___ [3]

Solution: *Sampling can never establish that no path exists, so no domain; it finds one with probability approaching one wherever one exists, so every domain. The intended practicality answer is "all": PRM/RRT can be reasonable choices for typical instances in each listed domain, including high-dimensional ones where uniform grids are too large. This is not a guarantee of fast planning for every instance; narrow passages, expensive collision checks and stringent accuracy or time requirements can make them impractical.*

Going further: An RRT has run for a long time without finding a path. Can you conclude that no path exists? How might a narrow passage explain the result?

Solution: *No. Failure after finitely many samples is not a proof of infeasibility. A narrow passage can have very little sampling volume, so the planner may simply not have sampled the configurations needed to connect through it.*

Problem 8 Cspace obstacles [1]

Gradescope: Which of these represent valid approaches to computing the Cspace obstacle for translating convex polygons moving among convex polygonal obstacles? Apply each construction to one obstacle at a time. Take the Minkowski difference of A and B to be $A - B = \{\, a - b \;:\; a \in A,\; b \in B \,\}$.

- √ **Convex hull of Minkowski difference of vertices of obstacle and robot**
- ◯ Convex hull of Minkowski sum of vertices of robot and obstacle
- √ **Star algorithm**
- ◯ Union of copies of the robot with its reference point placed at every point of the obstacle, including its interior, keeping the robot's orientation fixed
- √ **The same union using the "negated" robot, obtained by reflecting the robot through its reference point**

Solution: *The first, the star algorithm, and the union of translated copies of the negated robot. Terms: the robot's* reference point *is the point whose position we track, so placing the robot at x puts that point at x; the* Cspace obstacle *is the set of reference-point positions at which the robot overlaps the obstacle; the* Minkowski sum $A \oplus B = \{a + b\}$ *is every point of B slid to every point of A, and the difference $A - B = \{a - b\} = A \oplus (-B)$; the* negated robot $-R$ *is R reflected through its reference point; and the* star algorithm *builds $O \oplus (-R)$ for convex O and R by merging their edges in order of direction angle, without forming every vertex pair. With the robot's reference point as its local origin, the fourth construction gives $O \oplus R$ and the fifth gives $O \oplus (-R)$. The Cspace obstacle is the set of robot origins that put the robot in collision, which is $O - R = O \oplus (-R)$: the obstacle grown by the robot reflected through its origin. The order matters — $R - O$ is that set negated — and so does the reflection, which is what separates the first choice from the second and the fifth from the fourth. The first works because, for convex shapes, the sum is the convex hull — the smallest convex shape containing them — of the pairwise vertex sums, here differences; the second builds $O \oplus R$ instead.*

Going further: For a circular translating robot whose reference point is its center, would forgetting to negate the robot change the Cspace obstacle? Why might testing only this shape hide a bug?

Solution: *No: the centered disk satisfies $R = -R$, so $O \oplus R = O \oplus (-R)$. An asymmetric robot generally does not, so the same implementation can give the wrong collision region for another shape.*

Problem 9 Differential drive [1]

Gradescope: Consider a differential-drive (two wheel) rectangular robot. Select all that are true.

- ◯ This is a holonomic robot
- √ **This is a non-holonomic robot**
- ◯ Given two arbitrary (x, y, θ) poses in the plane (with no obstacles), this robot is not guaranteed to be able to move between them.
- √ **By rotating in place, moving in a straight line and rotating again, this robot can reach any (x, y, θ) pose in the plane (with no obstacles).**
- ◯ Even in the presence of obstacles, there is no advantage to moving backwards; going forwards is sufficient.

Solution: *The second and fourth. Differential drive is nonholonomic — it cannot translate sideways instantaneously — but the constraint is not integrable, so the reachable set is still all of (x, y, θ): rotate, drive, rotate. Backing up genuinely helps in clutter, which is why parallel parking exists.*

Going further: In an empty room, a differential-drive robot wants to end a short distance directly to its left with its original heading. Can it do so even though it cannot drive sideways? Describe a sequence of motions. How does this differ from a car?

Solution: *Yes. Turn left by $90°$, drive forward the required distance, then turn right by $90°$. A restriction on instantaneous velocity does not prevent reaching that final pose. A car cannot rotate in place and has a minimum turning radius, so it must turn while moving. With enough room (and reversing allowed), it can reach the same final pose using a sequence of forward and backward curved motions, as in parallel parking.*

Problem 10 Running times [1]

Gradescope: Mark all that are true.

- √ **The running time of complete motion planning algorithms grows exponentially (in the worst case) with the number of DOF of the robot.**

- ◯ The running time of complete motion planning algorithms grows exponentially (in the worst case) with the number of objects in the environment.
- √ **Probabilistically complete motion planning for a 7 DOF robot can typically be done in less than a milli-second, in the best implementations.**
- ◯ Probabilistically complete motion planning for a 7 DOF robot typically takes tens of seconds, even in the best implementations.

Solution: *The first and third. Complete planning is exponential in the dimension of the configuration space but only polynomial in the number of obstacles, and a good RRT implementation on a 7-dof arm is sub-millisecond on typical problems. "Typically" is important in the third and fourth: a naive implementation, or a query with a narrow passage in it, can take far longer than a millisecond.*

Going further: Two planning problems use the same 7-DOF arm and the same number of obstacles. One has a wide route to the goal; the other has only a narrow passage. Why might their RRT running times be very different?

Solution: *Dimension and obstacle count do not describe the geometry of free space. The narrow passage requires samples in a small, hard-to-reach region, so successful tree extensions can be much rarer.*

Problem 11 Grid-based planning [4]

Let's consider grid-based motion planning. The idea is that we make a uniform grid in configuration space, and mark each cell as occupied if any part of it is not free. Then we can use A* to plan a path from the starting grid cell to the goal grid cell, stepping between *neighboring* cells that share a face or a corner (so diagonal moves are allowed) and assigning a cost to each step that is the Euclidean distance between the two cells' centers. Read *finer* and *coarser* below as halving and doubling the cell size, so that the grids nest: every cell of a coarser grid is a block of whole cells of a finer one.

Gradescope:

11.1 Mark all that are true. [1]

- √ **For any motion planning problem that has a solution with some clearance — a path keeping a positive distance from every obstacle — there is a grid granularity fine enough so you can find a path.**
- ◯ If a path is found, it is guaranteed to be a shortest path in continuous Cspace, without restricting motion to the grid.
- ◯ If the search fails for some granularity then a solution might exist at a coarser granularity.
- √ **If the search fails for some granularity then a solution might exist at a finer granularity.**
- √ **The number of cells grows exponentially in the number of dimensions of the cspace.**
- ◯ The number of cells grows exponentially in the number N of divisions you make in each dimension of the cspace.

Solution: *The first, fourth and fifth. Marking a cell occupied when* any *of it is blocked is what makes refinement safe: splitting a free cell can only produce free cells, so a finer grid never loses a path that a coarser one had. Failure therefore rules out every coarser granularity (the third is false) while saying nothing about finer ones (the fourth). It also gives the first: once the cell diagonal is shorter than the clearance, every cell the path passes through lies within the clearance of it and so is free, and consecutive ones touch — the path's own cells are a grid path. The path A* returns is optimal on the grid graph, not in the continuous space — it can only step from cell to neighboring cell, and refining does not straighten the staircase out. With N divisions in each of d dimensions there are N^d cells: exponential in d, only polynomial in N.*

11.2 Now suppose one of the joints turns without limit, so that coordinate is an angle and its configuration space is a circle rather than an interval. We still divide it into N cells, now covering $[-\pi, \pi)$. For this joint, use the shortest angular difference when calculating step costs, including steps across the $-\pi/\pi$ boundary. Mark all that are true. [1]

- √ **The grid has the same number of cells as before.**
- √ **The cell at the top of the range and the cell at the bottom of it have to be neighbors.**
- √ **If we leave those neighbors out, the planner can report failure at every granularity on a problem that has a solution.**
- ◯ If we leave those neighbors out, the planner can return a path the robot cannot follow.
- ◯ A joint with limits, say $[-\pi/2, \pi/2]$, needs the same treatment.

Solution: *The first three. The joint's cspace is a circle, so it is the same N cells, but the two ends of the coordinate are the same place: the grid in that dimension is a ring, and the cells at either end of $[-\pi, \pi)$ are one step apart. (With every joint unlimited the cspace is a torus, and the grid with it.) Leaving those steps out cuts the ring open, and a problem whose only solution turns through $\pm\pi$ then fails at every granularity — resolution completeness is a property of the graph you build and not just of the cell size. It cannot make you unsafe, though: dropping edges only removes paths, so whatever does come back is still feasible. And a joint with limits needs the opposite treatment — its space really is an interval, and wrapping it would let the planner step from one limit to the other without passing through anything in between.*

11.3 A* also needs a heuristic. Let's use the straight-line Euclidean distance from a cell's center to the goal cell's center. For the first three choices, use a grid without wrap-around connections. Only the last choice uses the unlimited joint from the previous part. Mark all that are true. [1]

- √ **It is admissible.**
- √ **It stays admissible however the obstacles are arranged.**
- ◯ It stays admissible if instead every move to a neighboring cell, diagonal or not, costs 1, and the heuristic is measured in cell widths.
- ◯ It stays admissible for the unlimited joint of the previous part, if we go on computing the heuristic from the coordinates.

Solution: *The first two. A heuristic is* admissible *if it never overestimates the true remaining cost to the goal, which is what makes the first path A* finds optimal, and* consistent *if for neighbouring cells $h(a) \le c(a,b) + h(b)$, which lets A* finish with a cell the first time it expands it. Each step costs the distance it covers, so no route through the grid is shorter than the straight line: the heuristic is a lower bound, and being a distance it obeys the triangle inequality, so it is consistent as well as admissible and A* never reopens a cell. Ignoring the obstacles is what makes it a bound — they can only lengthen the true path, never shorten it — so no arrangement of them breaks it. It fails the moment the costs stop being distances: charging 1 a step, a diagonal buys $\sqrt{2}$ cells of progress for 1, and the heuristic can exceed the true cost. It fails again if the grid wraps and the heuristic does not — two cells either side of $\pm\pi$ are one step apart, and their coordinates differ by nearly 2π. Fixing the graph and forgetting the heuristic is the usual form of that bug.*

11.4 Take a 7-joint arm and divide each joint's range into $N = 100$ intervals. How many cells does the grid have? 10^{14} [1]

Solution: *$N^d = 100^7 = 10^{14}$. One bit per cell, just to record which cells are free, is 12.5 terabytes — before any search has run. This is the fifth item above with numbers in it, and it is why nobody plans for an arm on a grid.*

Going further: Suppose we mark a cell free whenever its center is collision-free, rather than requiring the entire cell to be free. Could the planner now return a path that collides? Give a simple example.

Solution: *Yes. A thin wall can lie between neighboring free cell centers, so the segment connecting them crosses the wall. Checking only cell centers does not certify the motion between them; that motion needs its own collision check.*

2 Computational questions

2.1 Optimization and differential IK

Problem 12 Introduction to QP [5]

The purpose of this exercise is to get you familiar with the basics of what an instance of an optimization problem is, as well as how to solve it.

An optimization problem is usually written as

$$\begin{aligned} \min_x \quad & f(x) \\ \text{s.t.} \quad & g(x) \leq 0, \\ & h(x) = 0 \end{aligned}$$

We call x the **decision variable**, $f(x)$ the **cost function**, $g(x) \leq 0$ an **inequality constraint**, and $h(x) = 0$ an **equality constraint**. We usually denote the optimal solution by x^*. Most of the times, the constraints are hard-constraints, meaning that they must be fulfilled by the optimal solution.

Drake offers a very good interface to many solvers using `MathematicalProgram`. Let's try to solve a simple problem using `MathematicalProgram`:

$$\begin{aligned} \min_x \quad & x^2 \\ \text{s.t.} \quad & x \geq 3 \end{aligned}$$

Before we start coding, what do you expect the answer to be? You should persuade yourself that the optimal solution is $x^* = 3$, since that is value at which minimum cost is achieved without violating the constraint.

The code for this problem is in `ps3/intro_to_qp.py`.

12.1 Run `test1()`. You should see that we were successful in getting the expected solution of $x^* = 3$.

12.2 Now write `scalar_example_interior` to minimize $x^2 - 2x$ subject to $-1 \leq x \leq 3$. These bounds replace the previous constraint $x \geq 3$. Run `test2()` to observe the result.

12.3 A particular class of problems that we want to focus on this problem are Quadratic Programs (QP), which can be solved very efficiently in practice (even on the order of kHz). The general formulation of these problems are defined as follows.

$$\begin{aligned} \min_x \quad & \tfrac{1}{2} x^T \mathbf{Q} x + c^T x \\ \text{s.t.} \quad & \mathbf{A} x \leq b, \\ & \mathbf{A}' x = b' \end{aligned}$$

where $\mathbf{Q}$ is a positive-semidefinite, symmetric matrix. Note that the cost is a quadratic function of the decision variables, while the constraints are all linear. This is what defines a convex QP.

Here is a simple QP, already worked out for you:

$$\begin{aligned} \min_{x_0, x_1, x_2} \quad & x_0^2 + x_1^2 + x_2^2 \\ \text{s.t.} \quad & \begin{pmatrix} 2 & 3 & 1 \\ 5 & 1 & 0 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \\ & \begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix} \leq \begin{pmatrix} 2 \\ 2 \\ 2 \end{pmatrix} \end{aligned}$$

This one is written for you as `vector_example` in the code; read it, then run `test3()` to see its solution. To conveniently write down constraints that are vector-valued, Drake offers `eq`, `le`, `ge` for elementwise constraints. It might take some time to learn the syntax of constraints. For a more in-depth introduction to `MathematicalProgram`, Drake's `MathematicalProgram` tutorial is very useful.

Now implement `solve_the_qp` to solve the following problem, and run `test4()`:

$$\begin{aligned} \min_{x_0,x_1,x_2} \quad & 2x_0^2 + x_1^2 + 4x_2^2 \\ \text{s.t.} \quad & \begin{pmatrix} 1 & 2 & 3 \\ 2 & 7 & 4 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \\ & |x| \leq \begin{pmatrix} 0.35 \\ 0.35 \\ 0.35 \end{pmatrix} \end{aligned}$$

The last constraint means that each absolute value `abs(x[i])` is at most `BOX[i] = 0.35`. To express this using linear constraints, there are two routes:

- Break the constraints down to two constraints that don't involve the absolute value.
- Drake offers `AddBoundingBoxConstraint` which you may use in your implementation.

Be sure you understand both approaches.

12.4 Run `test4()`, which solves your program and checks the answer against the constraints. [5]

Gradescope: The optimal solution is

$$x^* = (\ \underline{\textbf{0.1534}}\ ,\quad \underline{\textbf{-0.1005}}\ ,\quad \underline{\textbf{0.3492}}\ ),$$

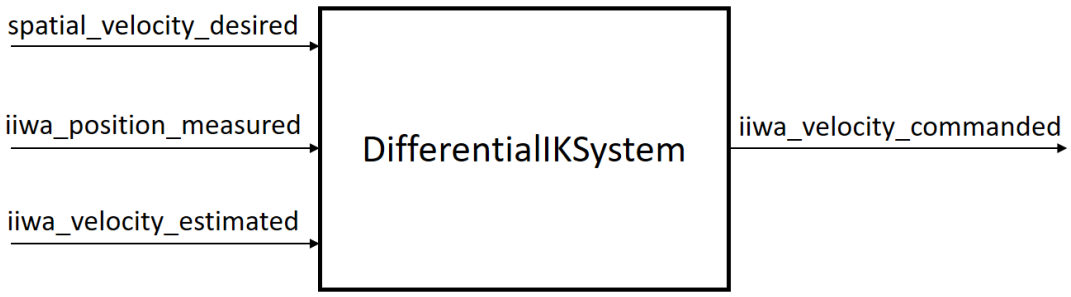
at which the cost is **0.5450**. (Give every entry to four decimal places.)

Solution: $x^* = (0.1534,\ -0.1005,\ 0.3492)$ *with cost* 0.5450.

Problem 13 Differential IK as optimization [8]

A differential IK controller converts a desired end-effector velocity into joint velocities. From Drake's system view of the world, how should we abstract this?

The controller is a `LeafSystem` with three input ports: `spatial_velocity`, `iiwa.position_measured`, and `iiwa.velocity_measured`. Its output port is `iiwa_velocity_command`. The measured position determines the Jacobian; the measured velocity is available to controllers that need it.



The diagram shows the conceptual signals; use the port names above when reading the supplied code.

Since we will go through several implementations of differential IK, `DifferentialIKSystem` takes a *function* that computes joint velocities from the Jacobian, the desired spatial velocity, and the rest of the robot's state. Swapping controllers is then a matter of passing a different function. `build_and_simulate` wraps the Drake plumbing around it:

- `HardwareStation`
- `DifferentialIKSystem`
- `ConstantVectorSource`, to command the desired velocity
- `Integrator`, to go from the controller's `iiwa_velocity_command` output to the station's `iiwa.position` input

All of this lives in `ps3/diffik_optimization.py`. Read the "code for students to be aware of" and "code for students to study" sections now.

13.1 Run `test1()`. It builds the whole diagram with `DiffIK_Zero`, a controller that commands zero velocity whatever it is asked for, so the manipulator should simply stay still.

13.2 In class we saw that to map a desired spatial velocity V^G to joint velocities we can use the relation [1]

$$V^G = J^G(q)\, v$$

and "invert" it with a Moore–Penrose pseudoinverse:

$$v = [J^G(q)]^+ V^G$$

That is `DiffIKPseudoInverse`, which we implemented in PS2. Run `test2()`, which asks the hand to move at 0.1 m/s in the $+x$ direction. (If Meshcat is slow to catch up, you can play back or scroll through the animation with its controls panel.)

Observe carefully what happens. Initially, the robot does move at 0.1 m/s in x. Verify that with respect to the colored world-frame axis (x is red).

Gradescope: Then what? Why? Submit one or two sentences in Gradescope.

Solution: *As the arm approaches a singular configuration, the pseudoinverse can demand large joint velocities to maintain the desired hand velocity. The simulation may slow, deviate from the requested motion, or move abruptly. Credit the connection to poor conditioning rather than a particular animation.*

13.3 How do we keep the benefits of the original method while improving the behavior near singularities? [1]

One idea from class is to hold to a **joint velocity limit**, which will at least stop the manipulator from doing something crazy. In the language of optimization, the problem we want to solve is

$$\begin{aligned} \min_{v} \quad & \|\mathbf{J}(q)v - V_d\|_2^2 \\ \text{s.t.} \quad & |v| \le v_{max} \end{aligned}$$

Implement `DiffIKQP`, transcribing and solving this program with `MathematicalProgram`, and run `test3()`.

Gradescope: What do you observe? Submit one or two sentences in Gradescope.

Solution: *The QP bounds each commanded joint speed. When accurate tracking would require exceeding those bounds, it accepts end-effector velocity error instead. Credit this tradeoff; the exact simulated motion can vary.*

13.4 Run `test5()`, which calls `DiffIKQP` on the fixed data at the bottom of the file (`J_G_GIVEN` and the values beside it) rather than on a simulation, so that everyone's answer is the same. It asks `DiffIKQP` for `V_G_DESIRED_FAST`, a command of 2 m/s in x and 1 m/s in y — far more than this arm can deliver from this configuration, so the velocity limit has to do some work. [4]

Gradescope: The first two joint velocities are (**3.0000**, **1.3128**); **2** of the seven joints end up at the 3.0 rad/s limit; and the largest joint speed the controller commands is $\max_i |v_i|$ = **3.0000** rad/s. (Four decimal places.)

Solution: *$v_0 = 3.0000$, $v_1 = 1.3128$, with joints 0 and 6 both pinned at the limit. Drop the bounding box constraint and the same command gives $v_0 = 3.6504$, $v_1 = 2.2629$ and a maximum joint velocity of 5.7852 rad/s, so this question does distinguish a controller that enforces the limit from one that does not — which the original command, `V_G_DESIRED_GIVEN`, did not: it leaves every joint below 0.29 rad/s and the constraint idle.*

13.5 In class we saw that on top of joint velocity constraints we can also constrain **joint positions** and **accelerations**, through a first-order Euler finite difference. Given v_{now} and q_{now}, we can ask for a v satisfying

$$\begin{aligned} \min_{v} \quad & \|\mathbf{J}(q_{now})v - V_d\|_2^2 \\ \text{s.t.} \quad & |v| \le v_{max} \\ & q_{min} \le q_{now} + hv \le q_{max} \\ & a_{min} \le (v - v_{now})/h \le a_{max} \end{aligned}$$

All of those constraints are written in joint space. But could we also end up with a linear constraint by imposing one in the *end-effector* space?

You must program a robot in a factory, and you would like a virtual wall that the robot is not allowed to put its hand through — space is tight and a human will be working there, or there are obstacles in the way.

Implement `DiffIKQP_Wall`, an optimization-based controller that minimizes $\|\mathbf{J}(q_{now})v - V_d\|_2^2$ subject to two constraints:

1. the joint velocity constraint $|v| \le v_{max}$, as before;
2. a constraint that the **position** of the end effector stay inside the box
   - `lower_bound = [-0.3, -1.0, 0.0]`
   - `upper_bound = [0.3, 1.0, 1.0]`

The current position of the hand is available as `p_now`, found by running forward kinematics. Constrain the gripper-frame origin in world coordinates using the last three Jacobian rows and prediction horizon $h = 0.004$ s. This horizon is not the controller's sampling period.

HINT: you can use a first-order approximation again, this time to ask where the hand will be next, given where it is now and how fast it is moving in end-effector space.

Run `test4()`, which draws the box and then commands a velocity that would carry the hand straight through it. Implemented correctly, the end effector stays inside the visualized box, almost as though the robot had a wall it could not cross. Small deviations can occur from linearization and simulation tracking errors. Note that constraining the *position* of the end effector is not the same as keeping the *entire* end effector inside the box — if part of it pokes out, that is fine. It is worth thinking about why.

13.6 Run `test6()`. [2]

Gradescope: With your `DiffIKQP_Wall`, the hand's x velocity is **-0.0885** m/s and its predicted next x position is **0.3000** m. (Four decimal places.) Use an absolute answer tolerance of 0.0001 in each quantity's units. The second value is a first-order prediction, not a measured position from the simulation.

Solution: *The hand's x velocity is -0.0885 m/s and its predicted next x position is 0.3000 m. `p_now` is at $x = 0.30035$, already a third of a millimetre outside the box, so the wall constraint is not a mild correction to the unconstrained answer: it is infeasible to go on as before, and the solver has to spend the step getting back inside.*

2.2 Optimization-based inverse kinematics

Problem 14 Inverse kinematics for door opening [11]

By using optimization-based inverse kinematics, we will solve a series of IK problems that can open a cupboard door. You can take a look at the `interactive_ik` notebook, which illustrates many of the concepts we go through in this question.

Our goal is to command the iiwa to open a cupboard door. We choose to open the door on the right. In `door_opening.py`, we provided you with setpoints ${}^{W}X^{G}(t)$ in the end-effector space, which gives us a nominal trajectory to reach and open the cupboard. "Nominal" refers to an ideal or baseline quantity. If you run `test1()` and open its Meshcat link, you will be able to visualize the trajectory. In the code, G is the gripper body frame, H is the handle frame, and ${}^{W}X^{G_{\text{grasp}}} = {}^{W}X^{H}\ {}^{H}X^{G}$, with ${}^{H}X^{G} =$ `X_handleG`. This trajectory divides the motion into the following segments:

- $0 \leq t \leq 5$: The end-effector linearly interpolates between the initial pose at $t = 0$, and the grasp pose required to grip the cylinder (${}^{W}X^{G_{\text{grasp}}}$ at $t = 5$), while having the gripper open.
- $5 \leq t \leq 6$: The end-effector stays still at ${}^{W}X^{G_{\text{grasp}}}$, and the gripper closes.
- $6 \leq t \leq 11$: The end-effector follows an arc of the handle as the door opens. The gripper remains closed.

We'll write code to convert this end effector trajectory to a joint-space trajectory using optimization!

The code for this problem is in `ps3/door_opening.py`.

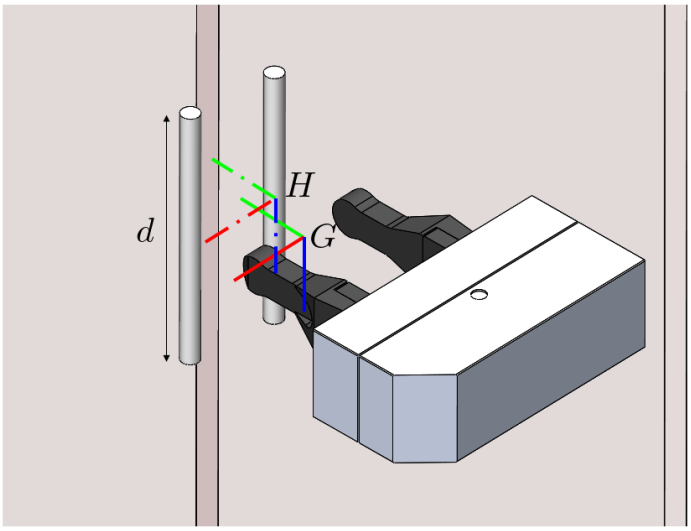
14.1 Start by running `test1()` to see the robot, door, and the target door trajectory.

14.2 Just for this question, we'll focus on just the grasping pose, and disregard the rest of the trajectory. We define H as the frame at the center of the handle, as shown below, and denote our decision variables as q (joint angles of the arm), and the forward kinematics function as [1]

$$ {}^{W}X^{G} = f(q) \ . $$

We will write ${}^{W}R^{G} = f_R(q)$ and ${}^{W}p^{G} = f_p(q)$ for the rotation and translation parts.

Recall one of the main ideas from the lecture: instead of having ${}^{W}X^{G} = f(q)$ be constrained to exactly ${}^{W}X^{G_{\text{grasp}}}$, the inverse kinematics problem can benefit by giving it freedom to pick from a set of values for ${}^{W}X^{G}$ which makes sense for our problem.



Let us parametrize ${}^{H}X^{G}$ with six degrees of freedom: xyz positions and roll-pitch-yaw rotation of frame G with respect to frame H. Three of them should be constrained to exactly their nominal grasp values; the other three can be given some slack. Select the three to constrain exactly. (Think about why the other three can be left free.) For this, let's ignore the length of the grippers. For this part only, use an idealized contact frame G at the midpoint of two point-contact fingers, rather than the gripper body frame used in the code. The handle lies along H's z axis, and the fingers close along G's x axis. At the nominal grasp, $(x, y) = (0, 0)$ and $(\text{roll}, \text{pitch}, \text{yaw}) = (\pi, 0, 0)$, using $R = R_z(\text{yaw})R_y(\text{pitch})R_x(\text{roll})$. Consider small variations about this nominal grasp and ignore the handle's ends. (Remember the x-y-z axes are represented in red-green-blue colors respectively.)

Gradescope:

- √ x **position**
- √ y **position**
- ◯ z position
- ◯ x orientation (roll)
- √ y **orientation (pitch)**
- ◯ z orientation (yaw)

Solution: *The x and y positions and pitch. The contact midpoint must lie on the handle axis, so its two transverse coordinates are zero. The closing axis must be perpendicular to the handle: $(R\hat{x}) \cdot \hat{z} = -\sin(\text{pitch}) = 0$, which fixes pitch to zero locally. Sliding along the handle, rotating about it, and rotating about the fingers' closing axis remain free in this idealized point-contact model. Finite fingers and the handle's ends would restrict those freedoms.*

14.3 For the rest of this problem, we want to enforce equality constraints (up to a small tolerance) on both the position and the orientation. This is because we will be solving IK along an entire trajectory, not just for the grasp pose. [1]

We can write our final optimization along the entire trajectory. For each pose I, we solve:

$$\begin{aligned} \min_{q} \quad & \|q - q_{nom}\|^2 \\ \text{s.t.} \quad & f_p(q) = {}^{W}p^{I} \\ & f_R(q) = {}^{W}R^{I} \end{aligned}$$

Recall that q_{nom} are the baseline joint angles we define and hold fixed throughout the optimization. Here q_{nom} = `Q_NOMINAL`; only the solver's initial guess changes between keyframes. What is the purpose of the quadratic cost term in our optimization formulation? Select all that apply.

Gradescope:

- √ **To bias the solver toward a preferred "natural" configuration when multiple solutions exist.**
- ◯ To ensure joint limits are respected.
- ◯ To guarantee that orientation constraints are satisfied exactly.
- √ **To penalize large joint displacements from q_{nom}.**

Solution: *The first and fourth. A cost expresses a preference; the other two are jobs for constraints, and a cost cannot guarantee either of them.*

We're getting ready to implement the optimization. Before you get started, go through these exercises to get an idea of how to implement the constraints in Drake's `InverseKinematics` class.

14.4 Read the documentation for the `AddOrientationConstraint` method in Drake. [1]
Note that `AddOrientationConstraint` implements an inequality constraint, which is okay for this problem if we set the theta bound to be small.

What are frames $\bar{A}$ and $\bar{B}$, and what is their purpose? Select all that apply.

Gradescope:

- ◯ Arbitrary "virtual" frames that do not need to exist in the plant
- √ **Frames existing in the `MultibodyPlant`**
- ◯ Task frames constrained to have angle difference $\theta \leq \theta_{bound}$
- √ **Reference frames such that ${}^{\bar{A}}R^{A}$ and ${}^{\bar{B}}R^{B}$ are constant throughout the optimization**
- ◯ Reference frames such that ${}^{\bar{A}}R^{A}$ and ${}^{\bar{B}}R^{B}$ may change throughout the optimization
- ◯ They are the frames which we wish to align
- √ **Use existing frames in the `MultibodyPlant` to express the task frames A and B in**

- √ **Allow you to define orientation constraints on arbitrary axes that are rigidly attached to bodies**
- ◯ Ensure that you only give the orientation of A and B relative to the world and gripper, respectively

Solution: *$\bar{A}$ and $\bar{B}$ are frames that exist in the plant, related to the task frames A and B by fixed rotations; A and B are what the θ_{bound} applies to. Their purpose is to let you express task frames in terms of plant frames, and so to constrain arbitrary axes rigidly attached to bodies.*

14.5 Read the documentation for the `AddPositionConstraint` method in Drake. Which of the following are true about the point Q, and what does the constraint enforce? [2]

Gradescope:

- √ **It is a point fixed relative to frame B**
- ◯ It is an arbitrary free-floating point in space
- ◯ It changes position relative to frame B during optimization
- √ **We aim to constrain its position in frame A**

A. That the coordinates of Q, expressed in frame B, remain between ${}^{A}p^{Q_lower}$ and ${}^{A}p^{Q_upper}$

B. That the distance between A and B is fixed

C. That the coordinates of Q, expressed in frame A, remain between ${}^{A}p^{Q_lower}$ and ${}^{A}p^{Q_upper}$

Solution: *Q is rigidly fixed to B — that is what makes it a handle on the body — and we bound where it lands when expressed in A. So the first and fourth checkboxes, and the third choice.*

Time to implement!

14.6 You will write two functions, using Drake's `inverse_kinematics` package.

First, `add_pose_constraint`, which asks an IK program for one frame to sit at one pose. Implement constraints on `ik` using `AddPositionConstraint` and `AddOrientationConstraint`. Note that these implement inequality constraints, which is okay if our error tolerance is small.

This one is deliberately written in terms of a frame, a pose and two tolerances, with nothing in it about the door: *the later IK problem in this pset imports it from here*, so the version you write now is the one that will be constraining the mobile manipulator. Keep it general.

Second, `create_q_knots`, which accepts a list of key frames `pose_lst` and converts them to joint coordinates. For each keyframe i:

- Call `add_pose_constraint` to put the gripper on that keyframe.
- Add a joint-centering cost on `Q_NOMINAL`.
- If `i==0`, set the initial guess to be `Q_NOMINAL` using `prog.SetInitialGuess`. Otherwise, set the initial guess to be the solution you obtained on the previous IK problem.

Note: When we define our constraints, we want to enforce the following tolerances

- Our position tolerance should be such that end effector matches the desired pose to within 1mm translation along each axis
- Our orientation tolerance should be such that the rotation is off by no more than `0.01*np.pi` radians

When you have implemented your function, run `test2()`. It smoothly interpolates (using a cubic spline—read about this if you haven't encountered the idea before) between the key-point joint configurations you found, and then executes it on the robot. Pose tolerances apply at the keyframes; joint-space interpolation need not maintain them between keyframes.

You should now see the robot following the trajectory to open the door!

14.7 **Gradescope:** When it looks good, run `test3()` and enter the returned joint angles for the final pose in the trajectory, to 4 decimal places. (The last 2 dimensions of q are the gripper joints, so we can ignore those.) Be sure you are using the proper tolerances for the orientation and position constraints [6]

(`0.01*np.pi` for orientation and `0.001` for position). The supplied `check_final_configuration` checks whether a returned seven-angle vector meets the final pose and joint limits; equivalent feasible answers need not match one reference vector.

Solution: *$q = (-1.2018,\ 1.3960,\ 1.3571,\ -0.6645,\ -2.8583,\ -2.0944,\ 1.5812)$, each ± 0.001 for comparison with this reference result.*

Problem 15 Inverse kinematics for a mobile manipulator [6]

In this problem, we'll solve inverse kinematics for one arm on a PR2 robot. This robot model has been modified to have convex collision geometries, and to simplify this problem, we have welded (fixed) several joints that are irrelevant to the kinematics of the arms.

The code for this problem is in `ps3/mobile_base_ik.py`.

15.1 Given a `RigidTransform X_WG`, we want to compute a robot configuration placing the left gripper at that pose. We use optimization to solve the IK problem, and we repeatedly solve the program with random initializations within a fixed attempt limit.

`solve_ik` is written for you: it builds the scene, sets up the `InverseKinematics` program with a cost that keeps the configuration near a nominal configuration, and runs the restart loop. It seeds NumPy's random number generator with `IK_SEED` before the first restart, so that a run repeats on your own machine; leave the seed alone. You write the two procedures it calls.

Implement `add_ik_constraints`, which adds the constraints the configuration has to satisfy:

- Put the gripper frame on the desired pose. You wrote this in the door-opening problem, and it is imported from there — call your `add_pose_constraint` rather than writing the constraints again.
  - The end effector should match the desired pose to within 1mm translation along each axis
  - The rotation should be off by no more than 1 degree. Note that this is a different tolerance from the door-opening problem's, which is why `add_pose_constraint` takes them as arguments.
- If `fix_base` is True, constrain the base pose (x, y, θ) to be equal to `base_pose`.
- Add a collision free constraint with `AddMinimumDistanceLowerBoundConstraint`. The minimum distance between any pair of collision geometries considered by Drake after collision filtering should be at least 1cm.

Implement `random_initial_guess`, which returns one configuration for the restart loop to start the solver from: draw each joint value uniformly within the robot's joint limits. The plant reports those bounds as arrays with one number per position, in the same order as the configuration vector:

- `plant.GetPositionLowerLimits()`
- `plant.GetPositionUpperLimits()`

Some of the positions — the base's x, y and θ — have no limit, and come back as $\pm\infty$; use the range $[-\pi, \pi]$ for those instead.

Run `test1()`, which fixes the base and runs all twenty restarts instead of stopping at the first success. It draws each configuration the solver stopped at — solved or not — and waits for you to press Enter, then reports how many succeeded and the cost of each solution.

15.2 The target is reachable from this base pose, but not every restart finds it. Roughly how many of the twenty succeed? [1]

Gradescope:

A. None.

B. Fewer than one in ten.

C. A substantial fraction, well short of all.

D. All twenty.

Solution: *A substantial fraction. Each restart starts from a random configuration, and a nonconvex program reached from a bad start can stall.*

15.3 `test1()` prints the cost of each solution, and they differ. What does the cost measure? [1]

Gradescope:

A. How far the gripper is from the target pose.
B. How long the solver took.
C. How far the configuration is from the nominal configuration.
D. How close the robot came to colliding.

Solution: *Distance from the nominal configuration, via* `AddQuadraticErrorCost`*. Reaching the target and avoiding collision are constraints, so every solution meets them equally; a high cost is a more contorted configuration, not a worse grasp.*

15.4 In `test1()`, where the base is fixed, what makes this case hard? Pick all that apply. [1]

Gradescope:

√ **It has 15 degrees of freedom**
◯ It has 7 degrees of freedom
◯ One of the arms is sticking out straight and can't be moved
√ **It has to avoid the shelves**

Solution: *A and D. With the base fixed there are fifteen positions to search over — the torso lift and seven joints in each arm — and only the left gripper's pose is constrained, so a random start can easily fall into a basin that holds no solution. The shelves cost restarts too: the minimum-distance constraint rules out configurations the arm could otherwise reach through. B counts just the left arm; the right arm and the torso are free variables the solver has to place. C is wrong on both halves: at the nominal configuration* both *arms stick straight out, since the elbow-flex stop is at zero, and neither is held there — only the head and the finger joints are welded shut.*

15.5 `test2()` tries four target positions, first from `test1()`'s base pose and then with the base free to drive. Which one can the arm reach only if the base moves? A failed IK search does not prove unreachability; use the robot's reach and joint limits to interpret the results. [1]

Gradescope:

A. $(-0.83, 0.18, 1.40)$
B. $(-0.83, 0.50, 1.40)$
C. $(-0.83, 1.30, 1.40)$
D. $(-0.83, 0.18, 2.30)$

Solution: $(-0.83, 1.30, 1.40)$*. With the specified fixed base, the shoulder-pan axis has horizontal coordinates* $(-1.35, 0.188)$*. The model's offsets from that axis to the gripper-palm origin have lengths* 0.100*,* 0.400*, and* 0.321 *m, so their sum* 0.821 *m is an upper bound on reach. The target's horizontal distance from the axis is* $\sqrt{0.52^2 + 1.112^2} \approx 1.228$ *m, already greater than this bound, even before accounting for height or joint limits. Moving the base removes that obstruction, and a successful IK solve supplies a feasible configuration.*

15.6 Which one is out of reach wherever the base goes? [1]

Gradescope:

A. $(-0.83, 0.18, 1.40)$
B. $(-0.83, 0.50, 1.40)$
C. $(-0.83, 1.30, 1.40)$
D. $(-0.83, 0.18, 2.30)$

Solution: $(-0.83, 0.18, 2.30)$*. The base moves in* x *and* y *only, so driving closer buys no height. The shoulder-pan axis is at height* 0.790675 *m with the torso lowered. Adding the maximum torso lift* 0.310 *m and the conservative arm reach* 0.821 *m gives an upper bound of* 1.921675 *m on the gripper-palm height, below* 2.30 *m.*

15.7 If we vary the x and y coordinates of the base pose for the fixed base case, we notice that sometimes IK fails to find a feasible solution. Which of the following base poses leads to no solution being found within the allowed attempts in your run? Use `test3()` to figure this out. [1]

Gradescope:

A. $[-1.23, 0.05, 0]$
B. $[-1.5, -0.2, 0]$
C. $[-1.8, 0.2, 0]$
D. $[-1.4, 0.3, 0]$

Solution: $[-1.8, 0.2, 0]$*: the base is too far from the shelf for the arm to reach the target. The other three all solve.*

2.3 Motion planning

Problem 16 RRT planning [6]

In our examples so far, we were able to get away with explicitly defining poses of interest and interpolating between them without any consideration of our environment. Unfortunately, with this approach, navigating cluttered environments while avoiding both collisions and system limitations is a major challenge. To address this, we'll use motion planning.

In this problem, we'll build two sampling-based motion planners. We'll start with the famous Rapidly-exploring Random Tree (RRT) algorithm, then extend it to the popular two-tree variant, RRT-Connect.

Optional background reading Kuffner & LaValle, "RRT-Connect: An Efficient Approach to Single-Query Path Planning" (ICRA 2000).

The code for this problem is in `ps3/rrt_planning.py`.

16.1 Study the code in the **Code for students to study** section: `iiwa_configuration_space`, `PlanningProblem`, and the `RRT_tools` class. Your planners are written in terms of these; be sure you understand each method before you use it.

16.2 Run `test1()` to see the problem instance. It shows the arm at q_{start}, pauses, and then shows it at q_{goal}, with the goal gripper pose drawn as a triad.

16.3 Implement the RRT algorithm, `rrt_planning` by implementing and using the helper `extend`. The path should reach the goal and pass the provided `edge_is_free` check on every edge.

For `extend`, the pieces are all in `RRT_tools`: `nearest_node`, `step_toward`, `edge_is_free` and `add_node`, and it reports one of the three `Status` values. For `rrt_planning` you also need `new_tree`, `sample_configuration` and `path_from_root`; the start and goal are `problem.start` and `problem.goal`, and `prob_sample_q_goal` is how often the goal itself is the sample.

Run `test2()`, which plans and then plays the path back in Meshcat. Do you notice the "RRT Dance"? Run it several times. Roughly how often does it succeed? Don't panic if it sometimes fails. Feel free to experiment with `max_iterations` and `prob_sample_q_goal`.

This simulation illustrates the limits of sampled planning checks. Contact can result from an inter-sample collision or from tracking error during execution. Compare any contact you observe with the finer recheck in part 16.6; passing that recheck still does not prove continuous collision freedom. A run with large contact forces or objects knocked over is not a successful execution: inspect the path and replan.

16.4 RRT-Connect builds on the basic RRT algorithm by growing two trees simultaneously: one rooted at the start and the other at the goal.

Implement the RRT-Connect algorithm, `rrt_connect_planning`, and run `test3()`. It plans and plays the path back the same way `test2()` does.

Reuse your `extend`. After successfully extending the first tree, repeatedly extend the second tree toward the new node until `Reached` or `Trapped`. One `tools` object serves both trees; a node's configuration is `node.value`, which is what you extend the other tree toward. When the trees

meet, `path_from_root` on each side gives a path from that tree's root to the meeting point, and `join_paths` splices them once the goal tree's path is turned around.

16.5 Run `test4()`, which runs both planners several times and prints the outer iterations and planning time for each trial. An RRT-Connect iteration may contain several extensions, so compare times as well as iteration counts.

16.6 **Gradescope:** `test2()` and `test3()` re-check the path they were given with `first_collision`, which walks it at a maximum per-joint spacing of `CHECK_STEP` — a tenth of the `EDGE_STEP` the planners used. Paths the planner accepted are sometimes reported `IN COLLISION` by this finer check. Explain why that happens, and say whether checking every `CHECK_STEP` instead would guarantee that the entire continuous path is collision-free. [1]

Solution: *The planner only ever evaluates collisions at the configurations `interpolate` visits, at most `EDGE_STEP` apart in each joint, so it is blind to anything thin enough to fit between two consecutive samples. Between two samples the whole arm sweeps, and near the bin and the mustard bottle one degree of joint motion is enough to carry the gripper a few millimetres into an obstacle. Nothing about the finer spacing is special: it finds more of these, but no finite spacing on its own proves a path is free, so this makes the check better, not correct. (Problem 17 shows what has to be added: a clearance margin at every sample.)*

16.7 Consider the following table, which indicates approximately how much, when the arm is fully outstretched, each joint would have to move (with the other joints fixed), to cause a 1 mm motion in the fingertip. The arm is at $q = 0$, pointing straight up, and the fingertip is the end of one finger pad, 6 cm off the arm's axis. What explains the pattern of the magnitudes of these numbers? [1]

joint	rotation (rad)	rotation (deg)
1	0.0167	0.955
2	0.00092	0.053
3	0.0167	0.955
4	0.00145	0.083
5	0.0167	0.955
6	0.00347	0.199
7	0.0167	0.955

Gradescope: Mark all that are true:

- ✓ **These values argue that we should use different spacing for different joints when interpolating in the RRT.**
- ◯ This same pattern of magnitudes will be the same no matter what configuration the robot is in.
- ◯ This is the Jacobian.
- ◯ This is the inverse Jacobian.

Solution: *Only A is true.*

A rotation $\delta\theta$ about an axis at distance r from the fingertip moves it by $r\,\delta\theta$, so each entry is 1 mm/r and the pattern is the pattern of lever arms. Joints 2, 4 and 6 have axes perpendicular to the outstretched arm, and their lever arms are the height of the fingertip above each joint: 1.09, 0.69 and 0.29 m. The earliest joint has the longest arm and needs the least rotation. Joints 1, 3, 5 and 7 have axes along the arm, so their only lever arm is the 6 cm finger offset; all four come out equal, and a point on the axis would not move at all.

A: True. With one joint-space step size, a step in joint 2 sweeps the fingertip about eighteen times as far as the same step in joint 7, so the interpolation is either too coarse for the early joints or needlessly fine for the late ones.

B: False. The lever arms depend on the configuration. Fold the elbow and joint 2's arm shrinks; turn the wrist and the twist joints' arms change.

C and D: False. Each entry is 1 mm divided by the norm of one column of the 3×7 translational Jacobian, moving one joint at a time. The Jacobian maps all seven joint velocities together to a fingertip velocity, and being 3×7 it has no inverse; a pseudoinverse would spread a 1 mm fingertip motion over all seven joints at once.

16.8 Consider the case where we let our RRT algorithm run forever, i.e. `max_iterations` grows without bound. If there is no path to the goal, will RRT warn you? If there is a path to the goal, how does the probability of RRT failing to find a path behave as the number of iterations goes to ∞? Assume a feasible path has positive clearance, sampling covers the free space, and local connections are checked correctly. [1]

Gradescope:

A. Yes / It approaches zero

B. No / It approaches zero

C. Yes / It remains greater than zero

D. No / It remains greater than zero

Solution: *No, and it approaches zero under the usual assumptions: a feasible path with positive clearance, sampling that can reach every open free region, and a suitable local extension/checking procedure. This is a limiting probability statement, not a finite-time guarantee. RRT does not certify nonexistence of a path.*

16.9 **Gradescope:** Upload the video of your robot following the RRT-Connect trajectory as an mp4 to Gradescope. Record the complete `test3()` playback with the scene visible. After the simulation finishes, use Meshcat's playback controls to return to the beginning. Start a screen recorder on that browser region, play the recording through once, and stop the screen recorder. Save or export as MP4; renaming another video format to .mp4 does not convert it. Open the exported file to verify that the complete motion is visible. The course file-size limit is 500 MB. [3]

2.4 Additional problems for 6.4212

Problem 17 Exact edge checking 6.4212 only

Our RRT decides whether a straight edge in configuration space is free by testing configurations at most `EDGE_STEP` apart in each joint, and the RRT problem ended with the observation that no fixed spacing *proves* an edge free. Schwarzer, Saha and Latombe ("Exact collision checking of robot paths") point out that a collision checker that reports a *distance*, rather than a yes or a no, can do better.

Consider one robot among static obstacles, and define two quantities. Here, collision-free refers to robot–obstacle collisions.

- The **clearance** $\eta(q)$ is the signed workspace distance between the robot, at configuration q, and the nearest obstacle. It is zero at contact and negative during penetration. The robot is collision-free at q exactly when $\eta(q) > 0$.
- A **motion bound** is a constant $\rho > 0$ such that, when the configuration moves along the straight line from any q to any q', no point of the robot travels farther than $\rho \, \|q - q'\|_2$ in the workspace.

Theorem. For endpoints with $\eta(q) > 0$ and $\eta(q') > 0$, if

$$\rho \, \|q - q'\|_2 \;<\; \eta(q) + \eta(q'),$$

then every configuration on the segment from q to q' is collision-free.

First evaluate both endpoint clearances and reject either endpoint if its clearance is nonpositive. When the test fails it says nothing either way, so the edge checker *bisects*: it computes η at the midpoint; if that is not positive it reports a collision; otherwise it applies the test to each half, recursively. The edge is free once every piece has passed.

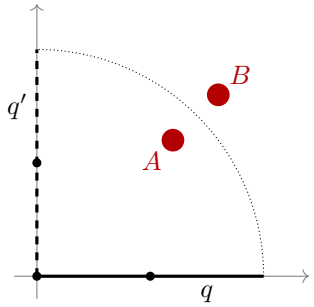
17.1 Prove the theorem. (Suppose some point p of the robot touches an obstacle partway along the segment. How far has p travelled by then, and how far does it still have to go?)

Solution: *At q, p is at least $\eta(q)$ from every obstacle, so it travels at least $\eta(q)$ to reach the contact. At q' it is at least $\eta(q')$ from the obstacle it touched, so it travels at least $\eta(q')$ more. That is a path of length at least $\eta(q) + \eta(q') > \rho \|q - q'\|_2$, contradicting the definition of ρ.*

17.2 A planar arm has two zero-thickness links of length 1 and configuration $q = (\theta_1, \theta_2)$, with θ_2 measured relative to link 1. Show that along a straight line in configuration space no point of the arm travels farther than $2\,|\Delta\theta_1| + |\Delta\theta_2|$, and conclude that $\rho = \sqrt{5}$ is a motion bound for this arm.

Solution: *A point p on link 2 has velocity $\dot\theta_1\,\hat z \times (p - o_1) + \dot\theta_2\,\hat z \times (p - o_2)$, where o_1 and o_2 are the joints, and $\|p - o_1\| \le 2$, $\|p - o_2\| \le 1$; so its speed is at most $2|\dot\theta_1| + |\dot\theta_2|$. On a straight line traversed in unit time $\dot q = \Delta q$ is constant, and integrating gives the bound. Points on link 1 move no farther than $|\Delta\theta_1|$. Finally $2|\Delta\theta_1| + |\Delta\theta_2| = (2, 1) \cdot |\Delta q| \le \sqrt{5}\,\|\Delta q\|_2$ by Cauchy–Schwarz.*

17.3 The arm sweeps from $q = (0, 0)$ to $q' = (\pi/2, 0)$: it stays straight and turns a quarter circle. The only obstacle is a round post of radius 0.1, which stands either at $A = (1.2, 1.2)$ or at $B = (1.6, 1.6)$. The clearances along the sweep are



θ_1	0°	22.5°	45°	67.5°	90°
η, post at A	1.100	0.549	−0.100	0.549	1.100
η, post at B	1.500	0.771	0.163	0.771	1.500

With the post at A, a fixed-step checker tests $\theta_1 = 0°, 10°, \ldots, 90°$. What does it report, and is it right?

Solution: *Free, and it is wrong. A is 1.70 from the base, so the arm overlaps the post only while θ_1 is within $\sin^{-1}(0.1/1.70) = 3.4°$ of 45°, and the samples at 40° and 50° straddle that.*

17.4 With the post at A, what does the bisecting checker do?

Solution: *$\rho\,\|q - q'\| = \sqrt{5}\,\pi/2 = 3.51$, which is not less than $1.1 + 1.1$, so the test fails. At the midpoint, 45°, $\eta = -0.1$: collision.*

17.5 With the post at B, at which configurations does the bisecting checker evaluate η, and what does it conclude?

Solution: *Free, after five evaluations. The whole segment fails, $3.51 \not< 3.0$, and $\eta(45°) = 0.163 > 0$. Each half has $\rho\,\|\Delta q\| = 1.76 \not< 1.5 + 0.163$, so it fails too, and $\eta(22.5°) = \eta(67.5°) = 0.771 > 0$. Each quarter has $\rho\,\|\Delta q\| = 0.88$, less than both $1.5 + 0.771$ and $0.771 + 0.163 = 0.93$, so all four are certified.*

17.6 As described, the bisection can run forever in exact arithmetic. Describe a segment on which it does. Then suppose the checker instead rejects any sampled configuration with $\eta \le \delta$, for a fixed margin $\delta > 0$: show that it never subdivides a piece shorter than $2\delta/\rho$, and conclude that checking at a *fixed* spacing ε is sound provided every sample has clearance greater than $\rho\,\varepsilon/2$. Use Euclidean configuration-space distance and sample both endpoints.

Solution: *A segment on which the robot grazes an obstacle, $\eta(q^*) = 0$, at a configuration q^* that is never a bisection point. By the definition of ρ, $\eta(q_a) \le \rho\,\|q_a - q^*\|$, so for any piece $[q_a, q_b]$ containing q^*, $\eta(q_a) + \eta(q_b) \le \rho\,\|q_a - q_b\|$ and the test fails, however short the piece. With the margin, every tested configuration has $\eta > \delta$, so a piece shorter than $2\delta/\rho$ has $\rho\,\|q_a - q_b\| < 2\delta < \eta(q_a) + \eta(q_b)$ and is certified. The same inequality certifies each pair of neighbors in a fixed-step check when $\delta = \rho\,\varepsilon/2$.*

Problem 18 Exact edge checking for the iiwa 6.4212 only

Now put the bisecting checker of the previous problem into the RRT. The code is `exact_edge_check.py`; it imports the scene, the planning problem and your `rrt_connect_planning` from `rrt_planning.py`.

18.1 Study `joint_radii` and `ClearanceChecker`. `joint_radii` returns, for each joint j, a bound r_j on the distance from that joint's axis to any point of the arm or gripper that the joint moves; it is the iiwa's version of the $(2, 1)$ of the planar arm. `ClearanceChecker.clearance` is $\eta(q)$, from Drake's `ComputeSignedDistancePairwiseClosestPoints`.

Implement `motion_bound`, which turns the r_j into ρ, and run `test1()`.

18.2 Implement `is_certified`, which is the theorem, and `exact_edge_is_free`, which is the bisection with the margin `DELTA`. Compute η once per configuration: pass the clearances at the two ends of a piece down the recursion with it. Also ask `collide` about each configuration you test; the last part explains why. `DELTA` is a rejection threshold at sampled configurations; the certificate guarantees positive clearance between samples, not a `DELTA` margin.

Run `test2()`. It plans with the fixed-step planner, as before, and then puts each edge of the path through your check.

18.3 Run `test3()`, which plans with your check inside RRT-Connect, by way of `ExactPlanningProblem`. How often do these paths fail the `CHECK_STEP` re-check? Why? Consider robot–obstacle collisions and self-collisions separately.

Solution: *For the modeled robot–obstacle pairs considered by the checker, the certificate proves positive clearance along an accepted edge, assuming a valid motion bound and reliable clearance calculations. It does not prove a `DELTA` margin between samples. Self-collision is checked only at sampled configurations, so a finer recheck can find a self-collision that was missed. There is no fixed expected failure frequency. Pairs excluded by the scene's collision filters are outside these guarantees.*

18.4 The last line `test1()` prints is the margin that would make the planner's fixed `EDGE_STEP` sound for robot–obstacle collisions, by the last part of the previous problem. Compare it with the clearance at q_{goal}. What would happen if we made the fixed-step planner sound by requiring that margin of every sample?

Solution: *It could not solve this problem: the margin is 4.5 cm and the goal has 1.3 cm of clearance, so the goal itself would be rejected. The bisecting checker subdivides where needed to certify positive robot–obstacle clearance, while rejecting sampled configurations with clearance at or below 5 mm. That sampled rejection threshold does not imply a continuous 5 mm clearance guarantee.*

18.5 `clearance` leaves out pairs of the arm's own links, and self-collision is caught only by the calls to `collide`, at the configurations that happen to be tested. The theorem does hold for such a pair, with the same ρ, if the motion of the descendant link is measured in the frame of the ancestor link. But at q_{start} the collision boxes of links 5 and 7 are 1.9 cm apart. What would including that pair in η do to the checker? What better bound is available for it?

Solution: *Wherever that gap stays near 1.9 cm, $\eta(q) + \eta(q')$ is at most 3.8 cm, and every edge is cut into pieces shorter than $0.038/1.94 = 0.02$ rad, about a degree, however far the arm is from everything else. Only joints 6 and 7 move link 7 relative to link 5, so the distance travelled in link 5's frame is at most $r_6|\Delta q_6| + r_7|\Delta q_7|$, which is far smaller than $\rho\,\|\Delta q\|$. This per-pair test is the one the paper actually uses.*

Upload one 1–2 page PDF with your answers to Questions 17 and 18 (no code) to Gradescope. This combined submission is worth 11 points.

3 What you should understand about the code you just produced

- In part 13.5 which constraints are active in the solution?
- In part 13.3 what would be another solution to the crazy behavior?

- Why does adding λI change the behavior of the pseudo-inverse near a singularity, and what does the robot actually do differently?
- When the QP for differential IK becomes infeasible, what should the controller do?
- What is the cost function doing in the door-opening IK, and what would change if you deleted it entirely?
- Why does the mobile base make the IK problem easier rather than harder, given that it adds decision variables?
- RRT-Connect grows two trees instead of one. Where does the speedup come from?
- What is the difference between a circular interpolation and a Piecewise Pose interpolation for the gripper in the door-opening example?
- When we use optimization-based IK on a whole trajectory, how do we initialize each solver call? Why?
- Does mobile-manipulation IK seem to be easier when the whole base is free to move or when it is fixed in a known feasible configuration? Why?
- In the RRT code, we have `EDGE_STEP = np.radians(1.0)`. This could cause a problem—would it be that we return a path that is actually not collision free, or that we fail to find a collision-free path if one exists? Give an example situation where this problem could arise.
- In RRT, what would happen if you set `GOAL_BIAS` to 0? to 1?
- How would the interpolation in the RRT method have to change if we had a joint that rotates freely all the way around?